

# UNIT 2 – DIVIDE AND CONQUER STRATEGY

## 1. GENERAL METHOD OF DIVIDE AND CONQUER

### 1.1 Introduction

**Divide and Conquer** is an important algorithm design technique used to solve complex problems by breaking them into smaller and simpler subproblems.

The basic idea is:

**Divide the problem → Solve the smaller problems → Combine their solutions**

Instead of solving a large problem directly, we divide it into smaller parts. Each smaller part is solved independently, usually using the same technique. Finally, the solutions of the smaller problems are combined to obtain the solution to the original problem.

Many important algorithms such as **Merge Sort, Quick Sort, and Binary Search** use the divide-and-conquer approach.

### 1.2 Three Main Steps

A divide-and-conquer algorithm generally consists of **three steps**.

#### 1. Divide

The original problem is divided into two or more smaller subproblems.

For example, suppose we have 8 elements:

```
[10, 20, 30, 40, 50, 60, 70, 80]
```

We can divide them into two parts:

```
[10, 20, 30, 40]      [50, 60, 70, 80]
```

These parts can again be divided:

```
[10,20] [30,40]      [50,60] [70,80]
```

## 2. Conquer

After dividing the problem, each smaller subproblem is solved.

If the subproblem is still large, it is divided again.

This process continues until the problem becomes sufficiently small that it can be solved directly.

For example:

```
[10,20]
```

can be divided into:

```
[10]  [20]
```

A single element is already sorted, so no further division is required.

## 3. Combine

After solving all the smaller problems, their solutions are combined to obtain the final solution.

For example:

```
[10] [20]  
↓  
[10,20]
```

and

```
[30] [40]  
↓  
[30,40]
```

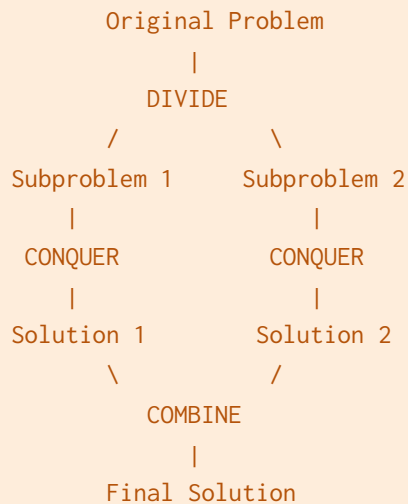
Then:

```
[10,20] + [30,40]  
↓
```

[10,20,30,40]

## 1.3 General Structure

The general idea can be represented as:



## 1.4 Example – Finding a Name in a Large Student List

Suppose a college has **10,000 students** and we need to find whether a particular student is present.

Instead of examining the entire problem at once, we can divide the list into smaller sections.

For example:

```

10,000 students
  ↓
5,000 + 5,000
  ↓
2,500 + 2,500 + ...
  ↓
smaller sections
  
```

We continue dividing until the required student can be located.

This demonstrates the basic idea of divide and conquer: **a large problem becomes easier when it is divided into smaller problems.**

## 1.5 Example – Merge Sort

Merge Sort is one of the best examples of divide and conquer.

Consider:

38 27 43 3

### Divide

38 27 | 43 3

Again:

38 | 27      43 | 3

### Conquer

Single elements are already considered sorted.

38      27      43      3

### Combine

Merge them in sorted order:

27 38      3 43

Finally:

3 27 38 43

Thus:

DIVIDE

38 27 43 3

↓

38 27      43 3

CONQUER

↓

COMBINE



3 27 38 43

## 1.6 Advantages of Divide and Conquer

1. **Breaks complex problems into smaller problems.**
2. Smaller problems are generally easier to understand and solve.
3. Many divide-and-conquer algorithms have efficient time complexity.
4. The technique is suitable for **recursive implementation**.
5. Some divide-and-conquer algorithms can be efficiently executed in parallel.
6. It is useful for designing algorithms for sorting, searching, matrix operations, and many other computational problems.

## 1.7 Disadvantages

1. Recursive calls may require additional memory.
2. Designing the correct divide and combine steps can sometimes be difficult.
3. For small problems, recursion may introduce unnecessary overhead.
4. Some problems cannot be efficiently solved using divide and conquer.

## 1.8 Important Examples

Common algorithms based on divide and conquer include:

- Binary Search
- Merge Sort
- Quick Sort
- Strassen's Matrix Multiplication

# 2. LINEAR SEARCH

## 2.1 Introduction

**Linear Search** is the simplest searching technique used to find an element in a list or array.

In linear search, elements are checked **one by one from the beginning until the required element is found or the end of the list is reached**.

It does not require the elements to be sorted.

For example:

Array:

10 25 30 45 50 60

Suppose we want to search for:

45

Linear search checks:

10 → Not found

25 → Not found

30 → Not found

45 → Found

Therefore, the element is found at the fourth position.

---

## 2.2 Example – Finding a Student's Name

---

Imagine a teacher has a list of students:

Rahul  
Anil  
Kiran  
Vamshi  
Ravi  
Sneha

Suppose the teacher wants to find **Vamshi**.

The teacher checks:

Rahul → No

Anil → No

Kiran → No  
Vamshi → Yes

The required name is found.

This is exactly how linear search works: **check each element sequentially until the required element is found.**

## 2.3 Working of Linear Search

Suppose the array is:

A = [15, 25, 10, 35, 40]

Search for:

35

The search proceeds as follows:

| Position | Element | Comparison      |
|----------|---------|-----------------|
| 0        | 15      | 15 $\neq$ 35    |
| 1        | 25      | 25 $\neq$ 35    |
| 2        | 10      | 10 $\neq$ 35    |
| 3        | 35      | 35 = 35 → Found |

Therefore, **35 is found at index 3.**

## 2.4 Algorithm for Linear Search

### Algorithm: Linear Search

**Step 1:** Start from the first element of the array.

**Step 2:** Compare the current element with the search element.

**Step 3:** If both are equal, return the position of the element.

**Step 4:** If they are not equal, move to the next element.

**Step 5:** Continue until the element is found or the array ends.

**Step 6:** If the end of the array is reached without finding the element, report that the element is not present.

---

## 2.5 Pseudocode

---

```
LinearSearch(A, n, key)
```

```
1. for i = 0 to n-1
2.     if A[i] == key
3.         return i
4. return -1
```

Here:

- **A** = array
  - **n** = number of elements
  - **key** = element to be searched
  - **i** = current position
  - **-1** = element not found
- 

## 2.6 Simple C Program

---

```
#include <stdio.h>

int main()
{
    int a[] = {10, 25, 30, 45, 50};
    int n = 5;
    int key = 45;
    int i, found = 0;

    for(i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            printf("Element found at position %d", i + 1);
        }
    }
}
```



```
        found = 1;
        break;
    }
}

if(found == 0)
    printf("Element not found");

return 0;
}
```

## Output

```
Element found at position 4
```

## 2.7 Time Complexity of Linear Search

The performance depends on the position of the required element.

### Best Case

The element is present at the first position.

```
[50, 10, 20, 30, 40]
  ↑
key = 50
```

Only **one comparison** is required.

Therefore:

**Best Case =  $O(1)$**

### Worst Case

The element is present at the last position or is not present.

```
10 20 30 40 50
           ↑
        key
```

All elements must be checked.

Therefore:

**Worst Case =  $O(n)$**

---

## Average Case

On average, approximately half of the elements may need to be examined.

Therefore:

**Average Case =  $O(n)$**

---

## 2.8 Advantages of Linear Search

---

- Very simple to understand.
  - Easy to implement.
  - Does not require sorted data.
  - Can be used for arrays, lists, and other sequential data.
  - Suitable for small datasets.
- 

## 2.9 Disadvantages

---

- Slow for large datasets.
  - May require checking every element.
  - Worst-case time complexity is  **$O(n)$** .
- 

## 2.10 When to Use Linear Search?

---

Linear search is useful when:

- The data is **small**.
- The data is **not sorted**.
- Simplicity is more important than search speed.
- The list is searched only occasionally.

## 3. BINARY SEARCH

---

### 3.1 Introduction

---

**Binary Search** is an efficient searching algorithm used to find an element in a **sorted array or sorted list**.

Unlike linear search, binary search does not check every element one by one.

Instead, it repeatedly divides the search range into **two halves**.

This is why binary search is an important example of the **divide-and-conquer technique**.

**Important condition:** Binary Search requires the data to be sorted.

---

### 3.2 Example – Dictionary

---

Imagine you want to find the word "**Computer**" in a dictionary.

You do not start from the first page and check every word.

Instead, you open the dictionary approximately in the middle.

Suppose the middle word is:

Network

Since **Computer** comes before **Network** alphabetically, you ignore the second half.

Now you search only the first half.

Again, you go to the middle.

You continue this process until you find **Computer**.

This is the basic idea of binary search.

---

### 3.3 Example – Student Roll Numbers

---

Suppose students' roll numbers are sorted:

101 102 103 104 105 106 107 108

We need to find:

106

Instead of checking:

101  
102  
103  
104  
105  
106

Binary search checks the middle element first.

Middle:

104

Since:

$106 > 104$

we ignore:

101 102 103 104

and search only:

105 106 107 108

Again, we check the middle.

This significantly reduces the number of comparisons.

---

## 3.4 Working of Binary Search

---

Consider:

```
A = [10, 20, 30, 40, 50, 60, 70]
```

Search for:

```
60
```

## Step 1

```
10 20 30 40 50 60 70
           ↑
          mid
```

Middle element:

```
40
```

Compare:

```
60 > 40
```

Therefore, the required element must be in the **right half**.

Ignore:

```
10 20 30 40
```

Search:

```
50 60 70
```

---

## Step 2

Middle element:

```
60
```

Compare:

```
60 = 60
```

Therefore:

```
Element Found
```

Only **two comparisons** were required.

---

## 3.5 Algorithm for Binary Search

---

**Step 1:** Ensure that the array is sorted.

**Step 2:** Set:

```
low = 0  
high = n - 1
```

**Step 3:** Calculate the middle position:

```
mid = (low + high) / 2
```

**Step 4:** Compare `A[mid]` with the search element.

**Step 5:** If:

```
A[mid] == key
```

the element is found.

**Step 6:** If:

```
key < A[mid]
```

search the left half.

```
high = mid - 1
```

**Step 7:** If:

```
key > A[mid]
```

search the right half.

```
low = mid + 1
```

**Step 8:** Repeat until the element is found or:

```
low > high
```

## 3.6 Pseudocode

```
BinarySearch(A, n, key)

1. low = 0
2. high = n - 1

3. while low <= high
4.     mid = (low + high) / 2

5.     if A[mid] == key
6.         return mid

7.     else if key < A[mid]
8.         high = mid - 1

9.     else
10.        low = mid + 1

11. return -1
```

## 3.7 Simple C Program

```
#include <stdio.h>

int main()
{
    int a[] = {10, 20, 30, 40, 50, 60, 70};
    int n = 7;
    int key = 60;

    int low = 0;
    int high = n - 1;
```

```
int mid;
int found = 0;

while(low <= high)
{
    mid = (low + high) / 2;

    if(a[mid] == key)
    {
        printf("Element found at position %d", mid + 1);
        found = 1;
        break;
    }
    else if(key < a[mid])
    {
        high = mid - 1;
    }
    else
    {
        low = mid + 1;
    }
}

if(found == 0)
    printf("Element not found");

return 0;
}
```

## 3.8 Time Complexity of Binary Search

The main advantage of binary search is that the search space is divided into half after every comparison.

Suppose there are:

1000 elements

After one comparison:

500

After two:



250

After three:

125

and so on.

The number of comparisons grows approximately as:

$$\log_2 n$$

Therefore:

### Best Case

The element is found at the middle during the first comparison.

**Best Case =  $O(1)$**

### Worst Case

The search continues by repeatedly dividing the array.

**Worst Case =  $O(\log n)$**

### Average Case

**Average Case =  $O(\log n)$**

---

## 3.9 Advantages of Binary Search

---

1. Much faster than linear search for large sorted datasets.
2. Reduces the search space by half in every step.
3. Worst-case complexity is  **$O(\log n)$** .
4. Efficient for large sorted arrays.

---

## 3.10 Disadvantages

---

1. Data must be sorted.
2. Maintaining sorted data can require additional work.
3. It is not suitable for every data structure.
4. Implementation is slightly more complex than linear search.

## 3.11 Linear Search vs Binary Search

| Feature             | Linear Search       | Binary Search           |
|---------------------|---------------------|-------------------------|
| Data requirement    | Sorted or unsorted  | Must be sorted          |
| Technique           | Sequential checking | Divide and conquer      |
| Best case           | $O(1)$              | $O(1)$                  |
| Worst case          | $O(n)$              | $O(\log n)$             |
| Simple to implement | Yes                 | Relatively more complex |
| Large datasets      | Less efficient      | More efficient          |
| Search method       | One by one          | Divide into halves      |

### Simple way to remember

#### Linear Search:

Check one by one.

#### Binary Search:

Check the middle and eliminate half.

## 4. MERGE SORT

### 4.1 Introduction

**Merge Sort** is a sorting algorithm based on the **Divide and Conquer** technique.

It divides an array into smaller parts until each part contains only one element. Then, it repeatedly merges these smaller parts in sorted order to produce the final sorted array.

The basic principle is:

**Divide the array → Sort the smaller arrays → Merge them**

Merge Sort is particularly important because its time complexity is  $O(n \log n)$  even in the worst case.

## 4.2 Example – Arranging Examination Marks

Suppose a teacher has marks of 8 students:

65 40 90 20 75 30 55 10

The teacher wants to arrange them in ascending order.

Instead of trying to sort all eight values at once, the problem can be divided:

65 40 90 20      75 30 55 10

Then:

65 40      90 20      75 30      55 10

Then:

65    40    90    20    75    30    55    10

Now the individual elements are merged in sorted order.

Eventually:

10 20 30 40 55 65 75 90

This is the basic working of Merge Sort.

## 4.3 Why is it called "Merge Sort"?

It is called **Merge Sort** because the algorithm repeatedly **merges two sorted portions** to produce a larger sorted portion.

For example:

```
[20, 40]    [10, 30]
```

Both parts are individually sorted.

Merge them:

```
10
10 20
10 20 30
10 20 30 40
```

Result:

```
[10, 20, 30, 40]
```

---

## 4.4 Working of Merge Sort

---

Consider:

```
38 27 43 3 9 82 10
```

### Step 1 – Divide

Divide into two parts:

```
38 27 43      3 9 82 10
```

Again:

```
38 27    43      3 9    82 10
```

Again:

```
38   27   43      3   9   82   10
```

Now each part contains one element.

## 4.5 Step 2 – Merge

Now start merging.

38 + 27

becomes:

27 38

Similarly:

3 + 9

becomes:

3 9

And:

82 + 10

becomes:

10 82

Now merge:

27 38 + 43

Result:

27 38 43

and:

3 9 + 10 82

Result:

3 9 10 82

Finally:

27 38 43  
+  
3 9 10 82

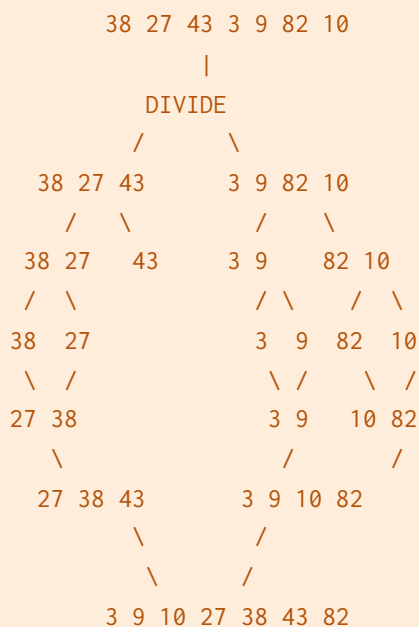
After merging:

3 9 10 27 38 43 82

Therefore, the sorted array is:

3 9 10 27 38 43 82

## 4.6 Merge Sort Diagram



The important thing students should understand is:

**The division phase breaks the array into individual elements, while the merging phase builds the sorted array.**

## 4.7 Algorithm for Merge Sort

### Merge Sort Algorithm

```
MergeSort(A, low, high)

1. If low < high
2.   mid = (low + high) / 2
3.   MergeSort(A, low, mid)
4.   MergeSort(A, mid + 1, high)
5.   Merge(A, low, mid, high)
```

The `Merge()` operation combines two sorted portions.

## 4.8 Simple C Program

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = 0;

    int temp[100];

    while(i <= mid && j <= high)
    {
        if(a[i] < a[j])
        {
            temp[k] = a[i];
            i++;
        }
        else
        {
            temp[k] = a[j];
            j++;
        }

        k++;
    }
}
```

```
while(i <= mid)
{
    temp[k] = a[i];
    i++;
    k++;
}

while(j <= high)
{
    temp[k] = a[j];
    j++;
    k++;
}

for(i = low, k = 0; i <= high; i++, k++)
{
    a[i] = temp[k];
}
}

void mergeSort(int a[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

int main()
{
    int a[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7;
    int i;

    mergeSort(a, 0, n - 1);

    printf("Sorted array: ");

    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}
```



## Output

Sorted array: 3 9 10 27 38 43 82

## 4.9 Time Complexity of Merge Sort

Merge Sort divides the array into two halves repeatedly.

For  $n$  elements, the number of levels is approximately:

$$\log_2 n$$

At each level, the merging operation processes approximately  $n$  elements.

Therefore:

$$T(n) = 2T(n/2) + O(n)$$

Using the Master Theorem:

$$T(n) = O(n \log n)$$

Therefore:

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n \log n)$   |
| Average Case | $O(n \log n)$   |
| Worst Case   | $O(n \log n)$   |

This is one of the major advantages of Merge Sort.

## 4.10 Space Complexity

Merge Sort requires an additional temporary array during merging.

Therefore, its auxiliary space complexity is:

$$O(n)$$

So students should remember:

Time  $\rightarrow O(n \log n)$

Space  $\rightarrow O(n)$

## 4.11 Advantages of Merge Sort

1. Guaranteed  **$O(n \log n)$**  time complexity.
2. Performs well for large datasets.
3. Suitable for sorting linked lists.
4. It is a **stable sorting algorithm** when implemented appropriately.
5. The divide-and-conquer structure makes it suitable for parallel processing.

## 4.12 Disadvantages of Merge Sort

1. Requires additional memory for merging.
2. For small arrays, simpler sorting algorithms can have less overhead.
3. The implementation is more complicated than simple algorithms such as Bubble Sort.
4. It does not generally perform in-place in its standard array implementation.

## 4.13 Important Points to Remember

### Divide and Conquer

Divide  $\rightarrow$  Conquer  $\rightarrow$  Combine

## Linear Search

Check elements one by one.

Complexity:

Best =  $O(1)$   
Average =  $O(n)$   
Worst =  $O(n)$

## Binary Search

Check middle → eliminate half → repeat

Requirement:

Array must be sorted.

Complexity:

Best =  $O(1)$   
Average =  $O(\log n)$   
Worst =  $O(\log n)$

## Merge Sort

Divide → Divide → Divide  
↓  
Merge  
↓  
Sorted Array

Complexity:

Best =  $O(n \log n)$   
Average =  $O(n \log n)$   
Worst =  $O(n \log n)$   
Space =  $O(n)$

# 5. QUICK SORT

## 5.1 Introduction

---

**Quick Sort** is an efficient sorting algorithm based on the **Divide and Conquer** strategy.

It was developed by **Tony Hoare**.

The basic idea of Quick Sort is to select one element from the array as a **pivot** and rearrange the remaining elements such that:

- Elements smaller than the pivot are placed on the **left**.
- Elements greater than the pivot are placed on the **right**.

The pivot is then in its correct position.

The same process is recursively applied to the left and right portions.

### Basic principle

Choose a pivot → Partition the array → Recursively sort the left and right parts

---

## 5.2 Why is it Called Quick Sort?

---

It is called Quick Sort because, in practice, it can be very fast for sorting large collections of data.

Its average time complexity is:

$$O(n \log n)$$

However, depending on how the pivot is selected, its worst-case complexity can become:

$$O(n^2)$$

---

## 5.3 Real-Time Example – Arranging Students by Marks

---

Suppose we have the marks:

```
65  40  90  20  75  30  55
```

Suppose we select:

```
65
```

as the pivot.

We want:

```
Elements smaller than 65 | 65 | Elements greater than 65
```

After partitioning:

```
40 20 30 55 | 65 | 90 75
```

Now we independently sort:

```
40 20 30 55
```

and:

```
90 75
```

Eventually:

```
20 30 40 55 65 75 90
```

Thus, a large sorting problem is divided into smaller sorting problems.

---

## 5.4 Important Terminology

---

### Pivot

The element selected for partitioning the array is called the **pivot**.

For example:

```
50  20  80  40  60
```

If we choose 50 as pivot:

```
50
↑
Pivot
```

## Partition

**Partitioning** means rearranging the elements around the pivot.

After partitioning:

```
Smaller elements | Pivot | Larger elements
```

For example:

```
20 40 | 50 | 80 60
```

The exact arrangement of elements on either side can vary.

## 5.5 Working of Quick Sort

Consider:

```
A = [50, 30, 70, 10, 40, 60, 20]
```

Let the last element be the pivot:

```
Pivot = 20
```

We rearrange the elements so that values smaller than 20 are placed before it and larger values after it.

Result:

```
10 | 20 | 50 30 70 40 60
```

Now 20 is in its correct position.

We recursively sort:

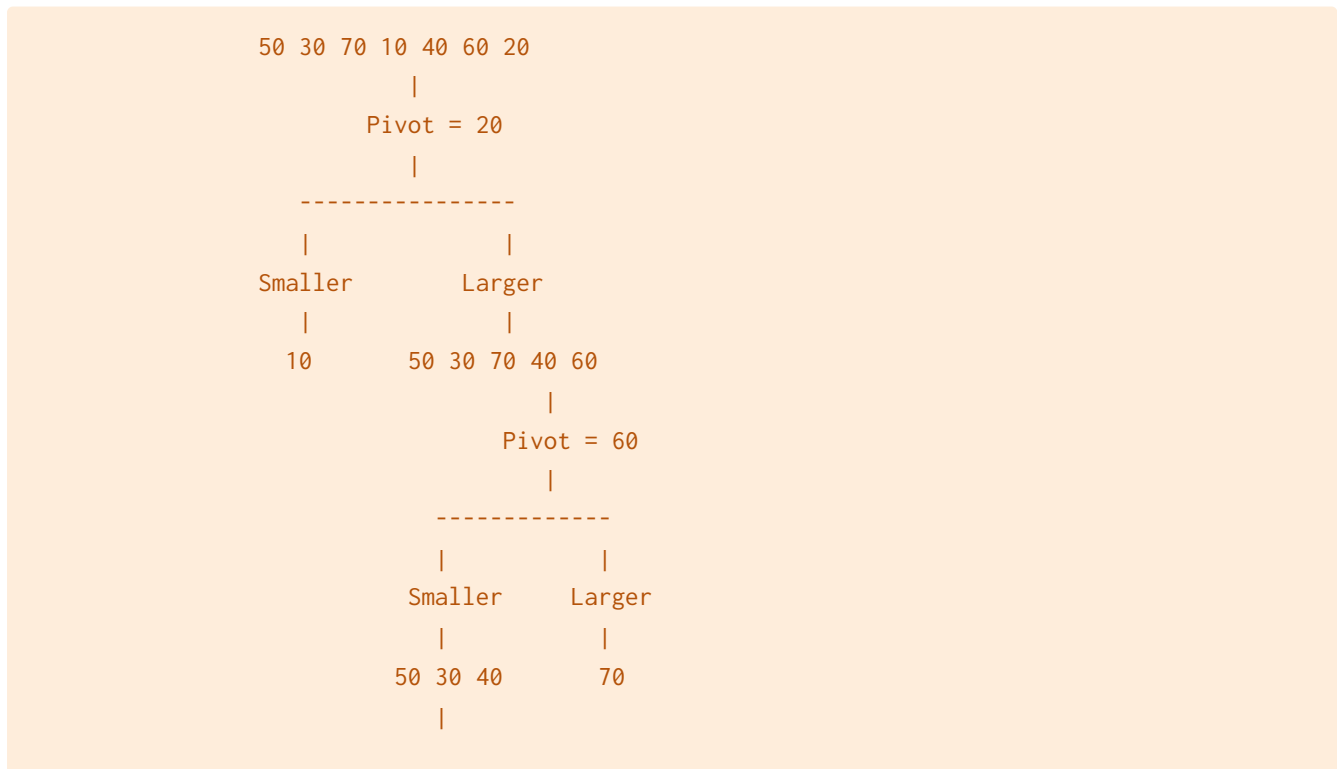
50 30 70 40 60

For the right side, choose another pivot, for example:

50 30 40 | 60 | 70

## Finally:

10 20 30 40 50 60 70



Continue  
|  
↓  
10 20 30 40 50 60 70

## 5.7 General Steps of Quick Sort

### Step 1 – Select Pivot

Choose an element as the pivot.

Common choices are:

- First element
- Last element
- Middle element
- Random element

### Step 2 – Partition

Rearrange the array around the pivot.

Smaller values | Pivot | Larger values

### Step 3 – Divide

The array is divided into two subarrays:

Left subarray      Pivot      Right subarray

### Step 4 – Conquer

Apply Quick Sort recursively to both subarrays.

### Step 5 – Combine



Unlike Merge Sort, Quick Sort does not require a separate merging operation.

Once the left and right partitions are sorted, the entire array is sorted.

This is an important difference.

---

## 5.8 Quick Sort Algorithm

---

```
QuickSort(A, low, high)

1. if low < high
2.     p = Partition(A, low, high)
3.     QuickSort(A, low, p - 1)
4.     QuickSort(A, p + 1, high)
```

The `Partition()` function determines the final position of the pivot.

---

## 5.9 Pseudocode for Partition

---

One common partition method uses the last element as the pivot.

```
Partition(A, low, high)

1. pivot = A[high]
2. i = low - 1

3. for j = low to high - 1
4.     if A[j] <= pivot
5.         i = i + 1
6.         swap A[i] and A[j]

7. swap A[i + 1] and A[high]

8. return i + 1
```

## 5.10 C Program – Quick Sort

---

```
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1;
    int j;

    for(j = low; j < high; j++)
    {
        if(a[j] <= pivot)
        {
            i++;
            swap(&a[i], &a[j]);
        }
    }

    swap(&a[i + 1], &a[high]);

    return i + 1;
}

void quickSort(int a[], int low, int high)
{
    if(low < high)
    {
        int p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main()
{
    int a[] = {50, 30, 70, 10, 40, 60, 20};
    int n = 7;
    int i;

    quickSort(a, 0, n - 1);

    printf("Sorted array: ");
```

```

for(i = 0; i < n; i++)
    printf("%d ", a[i]);

return 0;
}

```

## Output

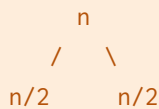
Sorted array: 10 20 30 40 50 60 70

# 5.11 Time Complexity of Quick Sort

The time complexity depends heavily on how well the pivot divides the array.

## Best Case

Suppose the pivot divides the array into approximately equal halves.



The recurrence is:

$$T(n) = 2T(n/2) + O(n)$$

Therefore:

$$T(n) = O(n \log n)$$

## Best Case:

$$O(n \log n)$$

## Average Case

In normal/random input conditions, Quick Sort generally performs in:

$$O(n \log n)$$

## Worst Case

Suppose the pivot is always the smallest or largest element.

For example:

10 20 30 40 50

If we always choose the last element as pivot:

10 20 30 40 | 50

Then:

10 20 30 | 40

and so on.

The division becomes highly unbalanced.

The recurrence becomes:

$$T(n) = T(n - 1) + O(n)$$

Therefore:

$$O(n^2)$$

## 5.12 Advantages of Quick Sort

1. Very efficient in practice.

2. Average complexity is  **$O(n \log n)$** .
3. Requires relatively little additional memory in its common in-place implementation.
4. Suitable for large arrays.
5. Uses divide-and-conquer.
6. Often performs faster than Merge Sort on arrays because of good cache behavior and lower constant factors.

## 5.13 Disadvantages

1. Worst-case complexity is  **$O(n^2)$** .
2. Performance depends on pivot selection.
3. Recursive implementation can cause stack overhead.
4. Standard Quick Sort is not stable.
5. Poor pivot selection can significantly reduce performance.

## 5.14 Merge Sort vs Quick Sort

| Feature           | Merge Sort         | Quick Sort                      |
|-------------------|--------------------|---------------------------------|
| Technique         | Divide and Conquer | Divide and Conquer              |
| Best Case         | $O(n \log n)$      | $O(n \log n)$                   |
| Average Case      | $O(n \log n)$      | $O(n \log n)$                   |
| Worst Case        | $O(n \log n)$      | $O(n^2)$                        |
| Extra array space | $O(n)$             | Usually $O(\log n)$ stack space |
| Stable            | Yes                | Usually No                      |
| Main operation    | Merge              | Partition                       |

## 6. STRASSEN'S MATRIX MULTIPLICATION

## 6.1 Introduction

---

**Strassen's Matrix Multiplication** is a divide-and-conquer algorithm for multiplying matrices.

It was developed by **Volker Strassen**.

The conventional method of multiplying two  $n \times n$  matrices requires:

$$O(n^3)$$

time.

Strassen's algorithm improves the asymptotic complexity to:

$$O(n^{\log_2 7})$$

which is approximately:

$$O(n^{2.807})$$

The key idea is to reduce the number of recursive multiplications.

---

## 6.2 Matrix Multiplication

---

Suppose we have two matrices:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

and:

$$B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

The conventional multiplication gives:

$$C = AB$$

where:

$$C = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

Notice that this requires **8 multiplications**:

ae  
bg  
af  
bh  
ce  
dg  
cf  
dh

Strassen's method cleverly calculates the result using only **7 multiplications**, at the cost of additional additions and subtractions.

## 6.3 Seven Strassen Products

Define:

$$M_1 = (a + d)(e + h)$$

$$M_2 = (c + d)e$$

$$M_3 = a(f - h)$$

$$M_4 = d(g - e)$$

$$M_5 = (a + b)h$$

$$M_6 = (c - a)(e + f)$$

$$M_7 = (b - d)(g + h)$$

Then calculate the resulting matrix:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

Therefore:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

---

## 6.4 Simple Numerical Example

---

Consider:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

and:

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Here:



$$a = 1$$

$$b = 2$$

$$c = 3$$

$$d = 4$$

$$e = 5$$

$$f = 6$$

$$g = 7$$

$$h = 8$$

Calculate:

**M1**

$$M_1 = (a + d)(e + h)$$

$$= (1 + 4)(5 + 8)$$

$$= 5 \times 13 = 65$$

**M2**

$$M_2 = (c + d)e$$

$$= (3 + 4)(5) = 35$$

**M3**

$$M_3 = a(f - h)$$

$$= 1(6 - 8) = -2$$

**M4**

$$M_4 = d(g - e)$$

$$= 4(7 - 5) = 8$$

**M5**

$$M_5 = (a + b)h$$

$$= (1 + 2)(8) = 24$$

**M6**

$$M_6 = (c - a)(e + f)$$

$$= (3 - 1)(5 + 6)$$

$$= 22$$

**M7**

$$M_7 = (b - d)(g + h)$$

$$= (2 - 4)(7 + 8)$$

$$= -30$$

Now calculate the result.

**C11**

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$= 65 + 8 - 24 - 30$$

$$= 19$$

**C12**

$$C_{12} = M_3 + M_5$$

$$= -2 + 24 = 22$$

**C21**

$$C_{21} = M_2 + M_4$$

$$= 35 + 8 = 43$$

**C22**

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

$$= 65 - 35 - 2 + 22$$

$$= 50$$

Therefore:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

---

## 6.5 Applications

---

Matrix multiplication is used in many areas of computing.

## Computer Graphics

Matrices are used for:

- Rotation
- Scaling
- Translation
- 3D transformations

For example, when a 3D game rotates a character, matrix operations are performed.

## Image Processing

Images can be represented using matrices. Matrix operations are used in:

- Image transformations
- Filtering
- Computer vision

## Scientific Computing

Large matrices are common in:

- Engineering
- Physics
- Numerical analysis
- Machine learning

## Machine Learning

Matrices are heavily used in:

- Neural networks
- Linear algebra operations
- Feature transformations

---

# 6.6 Complexity of Strassen's Algorithm

---

Strassen divides each matrix into four submatrices.

Instead of performing **8 recursive matrix multiplications**, it performs only **7**.

The recurrence is:

$$T(n) = 7T(n/2) + O(n^2)$$

Using the Master Theorem:

$$T(n) = O(n^{\log_2 7})$$

Since:

$$\log_2 7 \approx 2.807$$

Therefore:

$$T(n) = O(n^{2.807})$$

This is better asymptotically than:

$$O(n^3)$$

---

## 6.7 Advantages

---

1. Faster asymptotic complexity than conventional matrix multiplication.
2. Uses divide and conquer.
3. Useful for large matrices.
4. Important in scientific and numerical computing.

---

## 6.8 Disadvantages

---

1. More complicated than conventional matrix multiplication.
2. Requires more additions and subtractions.
3. Numerical stability can be an issue in some applications.
4. For small matrices, the overhead may make the standard algorithm preferable.
5. Practical performance depends on implementation and hardware.

## 7. SOLVING RECURRENCE RELATIONS

### 7.1 Introduction

Recurrence relations are very important in **algorithm analysis**, especially for recursive and divide-and-conquer algorithms.

A recurrence relation expresses the running time of an algorithm in terms of the running time of smaller inputs.

For example, suppose an algorithm divides a problem of size  $n$  into two subproblems of size  $n/2$  and spends  $n$  additional operations.

Its running time can be written as:

$$T(n) = 2T(n/2) + n$$

This is called a **recurrence relation**.

### 7.2 Why Do We Need Recurrence Relations?

Consider Merge Sort.

Merge Sort divides an array into two halves.

Therefore:

Problem size =  $n$

```

      n
     / \
    n/2 n/2
  
```

Each half is recursively sorted.

After that, the two sorted halves are merged.

The merge operation requires approximately  $n$  operations.

Therefore:

$$T(n) = 2T(n/2) + n$$

Recurrence relations allow us to determine the time complexity of recursive algorithms.

---

## 7.3 Components of a Recurrence Relation

---

A recurrence such as:

$$T(n) = 2T(n/2) + n$$

contains:

### Recursive part

$$2T(n/2)$$

This represents two recursive calls.

### Non-recursive work

$$+n$$

This represents the work performed outside the recursive calls.

---

## 7.4 Example 1 – Binary Search

---

Binary Search divides the problem into half.

Only **one** half is searched.

Therefore:

$$T(n) = T(n/2) + 1$$

Expanding:

$$T(n) = T(n/2) + 1$$

$$= T(n/4) + 1 + 1$$

$$= T(n/8) + 1 + 1 + 1$$

Continue until:

$$n/2^k = 1$$

Therefore:

$$2^k = n$$

Taking logarithm:

$$k = \log_2 n$$

Hence:

$$T(n) = O(\log n)$$

This explains why Binary Search is very efficient.

---

## 7.5 Example 2 – Merge Sort

---

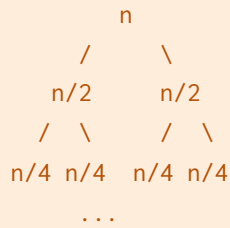
For Merge Sort:

$$T(n) = 2T(n/2) + n$$

At every level, approximately  $n$  work is performed.



The recursion tree looks like:



At each level:

Total work =  $n$

Number of levels:

$$\log_2 n$$

Therefore:

$$T(n) = n \log n$$

Hence:

$$O(n \log n)$$

## 7.6 Methods for Solving Recurrence Relations

There are several important methods.

### 1. Substitution Method

Also called the **iteration/substitution approach** in many introductory treatments.

### 2. Recursion Tree Method

The recurrence is represented as a tree and the cost at each level is calculated.

### 3. Master Theorem

A direct method for solving many divide-and-conquer recurrences.

These methods are important for your Unit 1/Unit 2 algorithm-analysis topics.

---

## 7.7 Substitution Method

---

Consider:

$$T(n) = T(n/2) + 1$$

Substitute repeatedly:

$$T(n) = T(n/4) + 2$$

Again:

$$T(n) = T(n/8) + 3$$

After  $k$  substitutions:

$$T(n) = T(n/2^k) + k$$

When:

$$n/2^k = 1$$

we get:

$$k = \log_2 n$$

Therefore:

$$T(n) = T(1) + \log_2 n$$

Thus:

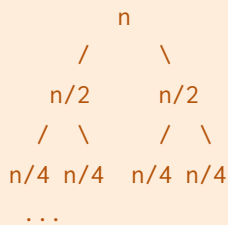
$$T(n) = O(\log n)$$

## 7.8 Recursion Tree Method

Consider:

$$T(n) = 2T(n/2) + n$$

Represent it as:



### Level 0

Cost:

$$n$$

### Level 1

There are two nodes:

$$n/2 + n/2 = n$$

### Level 2

There are four nodes:

$$n/4 + n/4 + n/4 + n/4 = n$$

Therefore, each level costs:

$$n$$

There are approximately:

$$\log_2 n$$

levels.

Hence:

$$T(n) = n \log n$$

Therefore:

$$O(n \log n)$$

---

## 7.9 Master Theorem

---

The Master Theorem is one of the most important tools for analyzing divide-and-conquer algorithms.

A recurrence is commonly written as:

$$T(n) = aT(n/b) + f(n)$$

where:

- $a$  = number of recursive subproblems
- $n/b$  = size of each subproblem
- $f(n)$  = cost of dividing and combining

---

## 7.10 Three Main Cases of Master Theorem

---

Compare:

$$f(n)$$

with:

$$n^{\log_b a}$$

---

## Case 1

If:

$$f(n) = O(n^{\log_b a - \epsilon})$$

for some  $\epsilon > 0$ , then:

$$T(n) = \Theta(n^{\log_b a})$$

---

## Case 2

If:

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

then:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

The common simple case is:

$$f(n) = \Theta(n^{\log_b a})$$

Then:

$$T(n) = \Theta(n^{\log_b a} \log n)$$

### Case 3

If:

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

and the regularity condition is satisfied, then:

$$T(n) = \Theta(f(n))$$

## 7.11 Master Theorem Example – Merge Sort

Consider:

$$T(n) = 2T(n/2) + n$$

Compare with:

$$T(n) = aT(n/b) + f(n)$$

Therefore:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ f(n) &= n \end{aligned}$$

Calculate:

$$n^{\log_b a}$$

$$= n^{\log_2 2}$$

$$= n$$

Therefore:

$$f(n) = \Theta(n)$$

This is Case 2.

Hence:

$$T(n) = \Theta(n \log n)$$

Therefore:

$$\boxed{\text{Merge Sort} = O(n \log n)}$$

---

## 7.12 Master Theorem Example – Binary Search

---

Consider:

$$T(n) = T(n/2) + 1$$

Here:

a = 1  
b = 2  
f(n) = 1

Calculate:

$$n^{\log_2 1} = n^0 = 1$$

Therefore:

$$f(n) = \Theta(1)$$

Using Case 2:

$$T(n) = \Theta(\log n)$$

Therefore:

$$\text{Binary Search} = O(\log n)$$

---

## 8. ANALYSIS OF DIVIDE AND CONQUER ALGORITHMS

---

### 8.1 Introduction

---

The analysis of an algorithm determines how efficiently an algorithm uses computational resources.

The two most important resources are:

1. **Time**
2. **Space**

When analyzing divide-and-conquer algorithms, we generally study:

- Number of subproblems
- Size of each subproblem
- Cost of dividing
- Cost of solving
- Cost of combining
- Recursion depth
- Time complexity
- Space complexity

---

### 8.2 General Form

---

A typical divide-and-conquer recurrence can be written as:



$$T(n) = aT(n/b) + f(n)$$

Where:

$a$

Number of subproblems.

$n/b$

Size of each subproblem.

$f(n)$

Cost of work performed outside recursive calls.

This gives us a systematic way to analyze the algorithm.

---

## 8.3 Example – Binary Search

---

Binary Search has:

- One subproblem
- Each subproblem has half the size
- Constant additional work

Therefore:

$$T(n) = T(n/2) + O(1)$$

Result:

$$O(\log n)$$

---

## 8.4 Example – Merge Sort

---

Merge Sort has:

- Two subproblems
- Each has size  $n/2$
- Merging requires  $O(n)$

Therefore:

$$T(n) = 2T(n/2) + O(n)$$

Result:

$O(n \log n)$

---

## 8.5 Example – Quick Sort

---

Quick Sort has two subproblems, but their sizes depend on the pivot.

### Balanced partition

Approximately:

$$T(n) = 2T(n/2) + O(n)$$

Therefore:

$$O(n \log n)$$

### Highly unbalanced partition

Approximately:

$$T(n) = T(n - 1) + O(n)$$

Therefore:

$$O(n^2)$$

This shows why pivot selection is important in Quick Sort.

## 8.6 Comparison of Important Divide-and-Conquer Algorithms

| Algorithm     | Recurrence             | Best/Average          | Worst          |
|---------------|------------------------|-----------------------|----------------|
| Binary Search | $T(n) = T(n/2) + O(1)$ | $O(\log n)$           | $O(\log n)$    |
| Merge Sort    | $2T(n/2) + O(n)$       | $O(n \log n)$         | $O(n \log n)$  |
| Quick Sort    | Depends on partition   | $O(n \log n)$ average | $O(n^2)$       |
| Strassen      | $7T(n/2) + O(n^2)$     | $O(n^{2.807})$        | $O(n^{2.807})$ |

## 8.7 Steps to Analyze a Divide-and-Conquer Algorithm

When you get an algorithm in an examination, follow these steps.

### Step 1 – Identify the input size

Let the input size be:

$$n$$

### Step 2 – Determine how the problem is divided

Ask:

Into how many subproblems is the original problem divided?

For example:

```
Binary Search → 1
Merge Sort → 2
Strassen → 7
```

---

### Step 3 – Determine subproblem size

For example:

```
Binary Search → n/2
Merge Sort → n/2
Strassen → n/2
```

---

### Step 4 – Calculate non-recursive work

Determine the work performed for:

- Dividing
  - Comparing
  - Merging
  - Partitioning
  - Combining
- 

### Step 5 – Write the recurrence

For example:

```
Binary Search:
T(n) = T(n/2) + 1
```

```
Merge Sort:
T(n) = 2T(n/2) + n
```

---

### Step 6 – Solve the recurrence

Use:

- Substitution method

- Recursion tree
- Master theorem

## Step 7 – Express the final complexity

For example:

$$O(\log n)$$

or:

$$O(n \log n)$$

## 8.8 Time Analogy for Understanding Analysis

Imagine a large college examination department has **10,000 answer scripts**.

### Method 1 – One person checks everything

One person checks every script.

Work is approximately:

$$O(n)$$

### Method 2 – Divide the work

The scripts are divided between two groups.

10,000  
↓  
5,000 + 5,000

Each group divides its work again.

5,000  
↓  
2,500 + 2,500

This continues.

The workload becomes easier to manage because a large problem has been divided into smaller problems.

This is the fundamental intuition behind **divide and conquer**.

---

## 8.9 Key Formula to Remember

---

For divide-and-conquer algorithms:

$$T(n) = aT(n/b) + f(n)$$

Remember:

a → Number of subproblems

b → Factor by which problem size decreases

f(n) → Work outside recursive calls

This single formula is extremely important for Unit 2.

---

## Exam Formulas

---

Keep this as a **one-page revision section**:

$$\textit{Binary Search} = O(\log n)$$

$$\textit{Merge Sort} = O(n \log n)$$

$$\text{Quick Sort Average} = O(n \log n)$$

$$\text{Quick Sort Worst} = O(n^2)$$

$$\text{Strassen} = O(n^{2.807})$$

$$T(n) = aT(n/b) + f(n)$$

---

## UNIT 2 – DIVIDE AND CONQUER STRATEGY

### IMPORTANT QUESTIONS

---

#### 2 MARKS QUESTIONS

---

1. Define Divide and Conquer strategy.
2. What are the three basic steps of Divide and Conquer?
3. Define Linear Search. State its worst-case time complexity.
4. Define Binary Search.
5. What is the basic requirement for applying Binary Search?
6. State the best-case and worst-case time complexities of Binary Search.
7. Define Merge Sort.
8. Write the recurrence relation for Merge Sort.
9. Define Quick Sort. What is a pivot element?
10. What is partitioning in Quick Sort?
11. State the best-case and worst-case time complexities of Quick Sort.
12. What is Strassen's Matrix Multiplication?

13. How many multiplication operations are required in Strassen's  $2 \times 2$  matrix multiplication?
  14. Define a recurrence relation and write its general form.
  15. Write the recurrence relations for Binary Search and Merge Sort.
- 

## 5 MARKS QUESTIONS

---

- Q1. Write the Binary Search algorithm and explain it with a suitable example.
  - Q2. Write the Quick Sort algorithm and explain it with a suitable example.
  - Q3. Explain the Merge Sort algorithm with a suitable example.
  - Q4. Explain the general method of Divide and Conquer with a suitable example.
  - Q5. Explain Linear Search with a suitable example and analyze its best, average and worst-case time complexities.
  - Q6. Explain recurrence relations used in Divide and Conquer algorithms with suitable examples.
- 

## 10 MARKS QUESTIONS

---

- Q1. Write and explain the Binary Search algorithm with a suitable example. Analyze its time complexity.

### Must cover:

- Definition
- Sorted-array requirement
- Algorithm
- Working
- Example
- Best case
- Average case
- Worst case
- Complexity



$$O(\log n)$$

---

**Q2. Write and explain the Quick Sort algorithm with a suitable example. Discuss its best, average and worst-case time complexities.**

**Must cover:**

- Definition
- Divide and Conquer
- Pivot
- Partitioning
- Algorithm
- Example
- Best case
- Average case
- Worst case

$$Best = O(n \log n)$$

$$Average = O(n \log n)$$

$$Worst = O(n^2)$$

---

**Q3. Explain the Merge Sort algorithm with a suitable example and derive its time complexity using a recurrence relation.**

**Must cover:**

- Divide
- Recursive division
- Merge operation
- Example
- Algorithm
- Recurrence

$$T(n) = 2T(n/2) + O(n)$$

Therefore:

$O(n \log n)$

---

**Q4. Explain Strassen's Matrix Multiplication algorithm with a suitable example and analyze its time complexity.**

**Must cover:**

- Conventional matrix multiplication
- Divide matrices into submatrices
- Seven multiplication expressions
- Result equations
- Numerical example
- Recurrence relation

$$T(n) = 7T(n/2) + O(n^2)$$

Therefore:

$O(n^{2.807})$

---

**Q5. Explain the recurrence relation for Divide and Conquer algorithms and discuss the methods used to solve recurrence relations with suitable examples.**

**Must cover:**

**General form**

$$T(n) = aT(n/b) + f(n)$$

**Methods**

1. Substitution/Iteration Method
2. Recursion Tree Method
3. Master Theorem

## Examples

Binary Search:

$$T(n) = T(n/2) + O(1)$$

$$O(\log n)$$

Merge Sort:

$$T(n) = 2T(n/2) + O(n)$$

$$O(n \log n)$$

---